

HR Data Warehouse & Analytics Project

A complete **Medallion Architecture** (Bronze → Silver → Gold) data warehouse built on **SQL Server**, transforming a single flat HR CSV extract into a governed, analysis-ready star schema, fully automated via **SQL Server Agent** and consumed by a 4-page **Power BI** report.

 SQL Server

 Power BI

 T-SQL

 Status

Table of Contents

- [Overview](#)
 - [Tech Stack](#)
 - [Architecture & Lineage](#)
 - [Repository Structure](#)
 - [Installation & Setup](#)
 - [Data Dictionary](#)
 - [Power BI Dashboard](#)
 - [Data Validation](#)
 - [Known Limitations](#)
 - [Roadmap](#)
 - [Contributors](#)
 - [License](#)
-

Overview

HR departments typically hold employee data — demographics, compensation, performance, recruitment source, tenure, termination reasons — in a single flat operational file that isn't prepared for analysis. This project converts that flat file into a governed, three-layer **data warehouse**, enabling reliable, repeatable, self-service HR reporting instead of ad hoc spreadsheet manipulation.

The warehouse follows the classic **Medallion Architecture**, entirely within a single SQL Server instance:

Layer	Description
 Bronze	Raw, unmodified copy of the source CSV, loaded via <code>BULK INSERT</code> . Every column is <code>VARCHAR(MAX)</code> — no type casting, no cleaning.
 Silver	Cleaned, standardized, type-cast, deduplicated, and conformed physical tables. Primary Keys are defined here. This is the single source of truth.
 Gold	Business-ready analytical SQL Views built directly on top of Silver — 5 dimension views + 1 fact view. No physical Gold tables and no surrogate keys.

Key implementation note: the Gold layer is implemented as **Views**, not physical tables. Views avoid duplicating data already governed in Silver, keep business logic centralized, and let Power BI query a simplified, presentation-ready structure that always reflects the current state of Silver — no separate Gold refresh/load step required.

Source data: [HRDataset_v14](#) — 311 employee records, 36 columns, one row per employee.

Consumption layer: a 4-page Power BI report — *Workforce, Turnover & Retention, Recruitment & Hiring*, and *Organization Insights* — connected to the Gold layer in Import mode.

Tech Stack

Technology	Role	Why It Was Chosen
Microsoft SQL Server	Hosts the <code>HR_Project</code> database and the <code>bronze</code> / <code>silver</code> / <code>gold</code> schemas	Industry-standard relational engine with native schema support, <code>BULK INSERT</code> , and Agent scheduling
SSMS	Development and execution environment	Native T-SQL authoring/debugging plus SQL Server Agent job management
T-SQL	All ETL logic	Keeps cleaning and modeling logic close to the data — no separate ETL engine needed at this scale
SQL Server Agent	Schedules the 5-step ETL pipeline	Native scheduling; enforces immediate-stop behavior on failure between interdependent steps

Technology	Role	Why It Was Chosen
Power BI Desktop	Semantic model + DAX + 4-page report	Enterprise-grade DAX for time-intelligence measures (turnover/retention) and native star-schema modeling
CSV / Excel	Raw source format	Matches a realistic HRIS flat-file export scenario
GitHub	Version control and delivery	Single source of truth for scripts, documentation, and the Power BI file

Architecture & Lineage

High-Level Architecture

```
flowchart LR
    A["📁 HR Dataset\nHRDataset_v14.csv"] --> B["🔥 Bronze Layer\nbronze.hr_data\n(raw, VARCHAR, no transformation)"]
    B --> C["🔧 Silver Layer\n8 cleaned & conformed tables\nPrimary Keys defined here"]
    C --> D["🔍 Gold Layer\nAnalytical SQL Views\n(no physical tables, no surrogate keys)"]
    D --> E["📊 Power BI\n4-page report"]
    D --> F["🔗 Ad-hoc SQL\nanalytical querying"]

    style A fill:#eeeeee,stroke:#888
    style B fill:#CD7F32,color:#fff
    style C fill:#C0C0C0,color:#000
    style D fill:#FFD700,color:#000
    style E fill:#F2C811,color:#000
    style F fill:#eeeeee,stroke:#888
```

Data Lineage – Silver → Gold Mapping

```
flowchart TD
    subgraph Silver
        emp[silver.emp]
        empstatus[silver.emp_status]
        position[silver.position]
        department[silver.department]
        manager[silver.manager]
        performance[silver.performance]
        recruitment[silver.recruitment]
    end
```

```

        snapshot[silver.hr_snapshot]
    end

    subgraph Gold["Gold Views"]
        v_dim_employee[gold.v_dim_employee]
        v_dim_position[gold.v_dim_position]
        v_dim_manager[gold.v_dim_manager]
        v_dim_performance[gold.v_dim_performance]
        v_dim_recruitment[gold.v_dim_recruitment]
        v_fact[gold.v_fact_hr_snapshot]
    end

    emp -->|LEFT JOIN| v_dim_employee
    empstatus -->|LEFT JOIN| v_dim_employee
    position -->|LEFT JOIN| v_dim_position
    department -->|LEFT JOIN| v_dim_position
    manager --> v_dim_manager
    performance --> v_dim_performance
    recruitment --> v_dim_recruitment

    snapshot -->|LEFT JOIN| v_fact
    v_dim_employee -->|LEFT JOIN| v_fact
    v_dim_position -->|LEFT JOIN| v_fact
    v_dim_manager -->|LEFT JOIN| v_fact
    v_dim_performance -->|LEFT JOIN| v_fact
    v_dim_recruitment -->|LEFT JOIN| v_fact

```

Note: Employee Status is embedded inside `gold.v_dim_employee` (not a separate view). Department is embedded inside `gold.v_dim_position` (not a separate view). `gold.v_dim_recruitment` has no `recruitment_id` — its business key is `RecruitmentSource`.

ETL Pipeline & Automation

Step	Script	Purpose	Depends On	Objects Created
1	<code>00_initialize_database.sql</code>	Create the database and the 3 schemas	—	<code>HR_Project</code> DB; <code>bronze</code> / <code>silver</code> / <code>gold</code> schemas
2	<code>01a_bronze_load.sql</code>	Load the raw CSV unmodified via <code>BULK INSERT</code>	00	<code>bronze.hr_data</code>

Step	Script	Purpose	Depends On	Objects Created
3	01b_bronze_profiling.sql	Run 15 diagnostic data-quality checks	01a	Query results only (no persisted objects)
4	02_silver_transformation.sql	Clean, type-cast, standardize, conform	01a	8 tables in the silver schema
5	03_gold.sql	Build the analytical Gold Views + validation queries	02	5 dimension views + 1 fact view
6	04_sql_agent_job.sql	Automate and schedule steps 1–5	00–03	One SQL Server Agent job, 5 steps, daily schedule

All 5 pipeline steps are chained in a single **SQL Server Agent job** (`Daily ETL`), scheduled once a day, with `on_success_action = go to next step` and `on_fail_action = quit reporting failure` on every step, so the job never continues silently after a failure.

⚠ **Known gap:** the delivered Agent job's Step 5 (`03_Gold`) still embeds the **legacy physical-table Gold DDL** (with surrogate keys), not the current View-based `03_gold.sql` . Update this step before relying on the scheduled job for a production rebuild.

Repository Structure

```
project root/
├─ README.md
├─ data/
│   ├── HRDataset_v14.csv
│   └─ README.md
├─ scripts/
│   ├── 00_initialize_database.sql
│   ├── 01a_bronze_load.sql
│   └─ 01b_bronze_profiling.sql
```

```

|   ├── 02_silver_transformation.sql
|   ├── 03_gold.sql
|   └── 04_sql_agent_job.sql
├── docs/
|   ├── architecture.jpeg
|   ├── data_flow.jpeg
|   ├── star_schema.jpeg
|   └── HR_DataWarehouse_Technical_Documentation.docx
└── power bi/
    ├── HR_Analysis_Dashboard_project.pbix
    ├── pbi_model_view.jpeg
    └── dashboard_1.png ... dashboard_4.png

```

Folder	Purpose
data/	The raw CSV file, exactly as downloaded, with a note on its source
scripts/	All T-SQL scripts, numbered in required execution order
docs/	Architecture, lineage, and star-schema diagrams, plus the full technical report
power bi/	The .pbix file and screenshots of each report page and the model view

Installation & Setup

Prerequisites

- Microsoft **SQL Server** (Developer or Express edition is sufficient for this data volume)
- **SQL Server Management Studio (SSMS)**
- **Power BI Desktop** (to open/refresh the report)

Steps

1. Clone this repository:

```
git clone https://github.com/rahmahamdy590-del/HR-DataWarehouse-Project.git
```

2. Set the Bulk Insert source path.

Open `scripts/01a_bronze_load.sql` and update the `BULK INSERT ... FROM` clause to point to your local copy of `data/HRDataset_v14.csv` :

```
BULK INSERT bronze.hr_data
FROM 'C:\path\to\your\HRDataset_v14.csv'
WITH (
    FORMAT = 'CSV',
    FIRSTROW = 2,
    FIELDQUOTE = '''',
    FIELDTERMINATOR = ',',
    ROWTERMINATOR = '0x0d0a',
    CODEPAGE = '65001',
    TABLOCK
);
```

⚠ The delivered script ships with a **hard-coded local Windows path**. This must be changed before running on a different machine — it has not been parameterized in the current implementation.

3. Run the scripts in SSMS, in exact order:

```
00_initialize_database.sql
→ 01a_bronze_load.sql
→ 01b_bronze_profiling.sql
→ 02_silver_transformation.sql
→ 03_gold.sql
```

4. (Optional) Automate the pipeline.

Deploy `04_sql_agent_job.sql` to an instance running SQL Server Agent to schedule the full pipeline end-to-end.

Remember to update job Step 5 (`03_Gold`) to call the current View-based `03_gold.sql` — see [Known Limitations](#).

5. Open the Power BI report.

Open `power bi/HR_Analysis_Dashboard_project.pbix` in Power BI Desktop, redirect the data-source connections to your `HR_Project` → `gold` schema (**Transform Data** → **Data Source Settings**), and refresh.

Data Dictionary

The Gold layer exposes **5 dimension views + 1 fact view**. None of these objects has a Primary Key, Foreign Key constraint, or surrogate/IDENTITY key — Gold is a pure presentation layer over Silver, where the real Primary Keys live.

`gold.v_dim_employee`

Built from `silver.emp LEFT JOIN silver.emp_status`. Includes Employee Status.

Column	Description
emp_id	Business key – employee ID
emp_name	Standardized full employee name
sex	Gender
dob	Date of birth
marital_desc	Marital status
citizen_desc	Citizenship status
hispanic_latino	Hispanic/Latino origin indicator
race_desc	Race/ethnicity description
state	U.S. state
empstatus_id	Business key – employment status ID
emp_status	Employment status classification (e.g., Active, Terminated)
termreason	Termination reason, if any
termd	Termination flag

gold.v_dim_position

Built from `silver.position LEFT JOIN silver.department`. Includes Department.

Column	Description
position_id	Business key – position ID
position	Job title
dept_id	Business key – department ID
dept_name	Department name

gold.v_dim_manager

Built from `silver.manager`.

Column	Description
manager_id	Business key – manager ID

Column	Description
manager_name	Manager's full name

gold.v_dim_performance

Built from `silver.performance`.

Column	Description
performance_id	Business key – performance rating ID
performance_score	Performance rating classification

gold.v_dim_recruitment

Built from `silver.recruitment`. No `recruitment_id` in the current implementation.

Column	Description
RecruitmentSource	Business key – recruitment channel (blank/NULL standardized to UNKNOWN)
FromDiversityJobFairID	Flag/ID referencing a diversity job-fair source

gold.v_fact_hr_snapshot (Fact View)

Built from `silver.hr_snapshot`, `LEFT JOIN` to each dimension view above. Grain: one row per employee HR-snapshot record.

Column	Source	Description
emp_id	v_dim_employee	Employee business key
emp_name	v_dim_employee	Employee name
position_id	v_dim_position	Position business key
position	v_dim_position	Job title
dept_id	v_dim_position	Department business key
dept_name	v_dim_position	Department name
manager_id	v_dim_manager	Manager business key

Column	Source	Description
manager_name	v_dim_manager	Manager name
performance_id	v_dim_performance	Performance business key
performance_score	v_dim_performance	Performance rating
RecruitmentSource	v_dim_recruitment	Recruitment channel
FromDiversityJobFairID	v_dim_recruitment	Diversity job-fair flag/ID
salary	silver.hr_snapshot	Annual salary
absences	silver.hr_snapshot	Total absences
dayslatelast30	silver.hr_snapshot	Days late in the last 30 days
engagementsurvey	silver.hr_snapshot	Engagement survey score (0–5)
empsatisfaction	silver.hr_snapshot	Satisfaction score (1–5)
specialprojectscount	silver.hr_snapshot	Count of special projects
date_hiring	silver.hr_snapshot	Hire date
date_termination	silver.hr_snapshot	Termination date
LastPerformanceReview_Date	silver.hr_snapshot	Date of the last performance review

Power BI Dashboard

A 4-page report connected to the **Gold layer only**, in **Import mode**:

Page	Business Question	Key KPIs
Workforce	What does our current workforce look like?	311 employees · 56.6% female / 43.4% male · avg. salary \$69.02K · avg. satisfaction 3.89

Page	Business Question	Key KPIs
Turnover & Retention	How much attrition are we seeing, and why?	Turnover rate 32.80% · Retention rate 67.20% · Highest-turnover dept: Production
Recruitment & Hiring	Which recruitment channels are working?	Total hires 311 · Top source: Indeed · Dept. with most hires: Sales
Organization Insights	How is the org structured, and how are managers performing?	Most common role: Production Technician I · Largest team: 22

The model uses a role-playing **Calendar/Date** dimension (defined inside Power BI) to support Hire Date, Termination Date, and Last Performance Review Date analysis via `USERRELATIONSHIP`.

Data Validation

`03_gold.sql` includes the following built-in validation checks:

1. **Silver-to-Gold row-count reconciliation** — confirms the fact view isn't silently dropping or duplicating rows.
2. **Missing Employee relationship check** — flags any fact row that can't resolve to an employee.
3. **Missing Position relationship check** — flags any fact row that can't resolve to a position/department.
4. **Employee Status consistency check** — flags any `EmpStatusID` mapping to more than one raw status label.
5. **Termination-date consistency check** — flags any non-active employee missing a termination date.

Known Limitations

-  **Hard-coded file path:** the `BULK INSERT` source path in `01a_bronze_load.sql` is a fixed local Windows path and must be edited before running elsewhere.
-  **Outdated Agent job step:** the SQL Server Agent job's `03_Gold` step still runs the legacy physical-table DDL instead of the current View-based script.
-  **No SQL-level Date dimension:** the Calendar/Date table exists only inside the Power BI model, not as a `gold` SQL view.

-  **Static source vs. daily schedule:** the source is a one-time CSV extract, so the current daily Agent schedule does not actually capture new data.
-  **No containerization:** the project currently runs on a local/on-prem SQL Server instance; Docker packaging is a proposed future improvement, not part of the current implementation.

Roadmap

- ☐ Update Agent job Step 5 to run the current Gold Views script
- ☐ Build a native SQL `gold.v_dim_date` calendar dimension
- ☐ Parameterize the Bulk Insert source path
- ☐ Containerize SQL Server + pipeline (Docker)
- ☐ Move to incremental loading instead of full reload
- ☐ Publish the report to the Power BI Service with scheduled refresh
- ☐ Add Row-Level Security (RLS) by department
- ☐ Migrate to Azure SQL Database / Microsoft Fabric

Contributors

Name	Role
Rahma Hamdy	Team Lead · Gold Layer
Angham Atef	Database creation & Bronze load layer
Samah Ali	Bronze data-quality profiling
Hend Mostafa	Silver Layer
Nermeen Khaled	Power BI Dashboard
Raghad Atef	Diagrams

License

This project is provided for educational and portfolio purposes. Add a license of your choice (e.g., MIT) if you plan to distribute or reuse this code.